

SAA-C02 — 100 Trick Questions: IAM & Everything Related

Difficulty: Hard | Focus: IAM, SCP, Resource Policies, Federation, STS, Permissions Boundaries, and more

SECTION 1: IAM Fundamentals & Evaluation Logic (Q1–Q15)

Q1. A developer has an IAM user with an explicit **Allow** on `s3:GetObject`. The bucket policy on the target S3 bucket has an explicit **Deny** for that IAM user. The user attempts to download an object. What happens?

- A) Access is allowed because IAM policy takes precedence
- B) Access is denied because explicit Deny always wins
- C) Access is allowed because Allow + Allow = Allow
- D) Access depends on the S3 bucket owner

Answer: B Reasoning: In AWS policy evaluation, an explicit **Deny** always overrides any **Allow**, regardless of where the Allow originates. Even if the IAM identity policy allows the action, the resource policy's explicit Deny wins. This is the foundational rule of IAM evaluation logic.

Q2. An IAM user belongs to two groups. Group A has a policy that **Allows** `ec2:TerminateInstances`. Group B has a policy that **Denys** `ec2:TerminateInstances`. The user also has an inline policy that **Allows** `ec2:TerminateInstances`. What is the effective permission?

- A) Allow — two Allows outweigh one Deny
- B) Deny — explicit Deny wins
- C) Allow — inline policy takes precedence over group policies
- D) Depends on policy evaluation order

Answer: B Reasoning: Regardless of how many Allow statements exist (inline or group), a single explicit **Deny** anywhere in the chain results in a final **Deny**. AWS does not count votes; it looks for any explicit Deny first.

Q3. A cross-account IAM role exists in Account B. An IAM user in Account A assumes that role. The IAM role in Account B has an **Allow** on **s3:PutObject** for a specific S3 bucket in Account B. However, the IAM user in Account A has **no permissions policy** attached. Can the user assume the role?

- A) No — the user must have **sts:AssumeRole** permission in Account A
- B) Yes — cross-account roles bypass identity policies
- C) No — the user must also have **s3:PutObject** in Account A
- D) Yes — trust policies are sufficient

Answer: A Reasoning: For cross-account role assumption, **two things** must align: (1) the **trust policy** on the role in Account B must allow Account A's user to assume it, AND (2) the **identity policy** of the IAM user in Account A must explicitly grant **sts:AssumeRole**. Without **sts:AssumeRole** in Account A, the call fails.

Q4. An IAM policy contains both a **Deny** with a **Condition** and an **Allow** without a condition for the same action. When the condition is **not met**, what is the result?

- A) Deny — the Deny statement still applies
- B) Allow — the condition was not met, so Deny is skipped
- C) Deny — any Deny statement blocks access
- D) Depends on evaluation order

Answer: B Reasoning: A conditional **Deny** only takes effect when the condition evaluates to true. If the condition is **not met**, the Deny statement is effectively skipped. The remaining **Allow** then applies. This is a common trick — a Deny with a condition is NOT an unconditional Deny.

Q5. Which of the following is true about IAM **managed policies**?

- A) They can be attached to users, groups, roles, and resources
- B) They are account-specific and cannot be shared across accounts

- C) They can be attached to multiple identities but cannot be edited after creation
- D) They can be attached to users, groups, and roles, but not resources

Answer: D Reasoning: Managed policies (AWS-managed or customer-managed) are **identity-based policies** and can be attached to IAM users, groups, and roles. They **cannot** be attached directly to AWS resources (like S3 buckets) — those use **resource-based policies**. They can be edited after creation, unlike inline policies which are embedded.

Q6. An IAM role has a permissions boundary that allows only `s3:*`. The role's permission policy allows `s3:*` and `ec2:*`. What actions can the role perform?

- A) `s3:*` and `ec2:*`
- B) Only `s3:*`
- C) No actions — boundaries conflict with policies
- D) Only `ec2:*`

Answer: B Reasoning: A **permissions boundary** defines the maximum permissions an identity can have. The **effective permissions** are the intersection of the identity policy AND the permissions boundary. Even though the role's policy grants `ec2:*`, the boundary does not include it, so `ec2:*` is blocked.

Q7. A permissions boundary on an IAM user allows `s3:GetObject`. The user's identity policy allows `s3:PutObject`. An S3 bucket policy explicitly allows the user `s3:PutObject`. What can the user do?

- A) `s3:GetObject` and `s3:PutObject`
- B) Only `s3:GetObject`
- C) Only `s3:PutObject`
- D) Neither — conflicting policies result in implicit Deny

Answer: B Reasoning: For identity-based actions, the effective permission = intersection of (identity policy) AND (permissions boundary). The boundary only allows `s3:GetObject`, so `s3:PutObject` is blocked **even if** the bucket policy allows it. The bucket policy cannot override a permissions boundary on an identity.

Q8. What is the default behavior for a brand-new IAM user with no policies attached who tries to access any AWS service?

- A) Full access — root account permissions apply by default
- B) Read-only access to IAM and billing
- C) Implicit Deny — no access to anything
- D) Access to services in the default VPC only

Answer: C Reasoning: IAM follows a **default deny** model. A new IAM user with no attached policies has **no permissions whatsoever** (implicit deny). They cannot access any service, including IAM or billing, unless explicitly granted via a policy.

Q9. An IAM group named **Developers** is attached to a policy. A new IAM user is added to the group. Which statement is true?

- A) The user inherits the policy and it becomes an inline policy for that user
- B) The user inherits permissions from the group's policy
- C) The user must explicitly accept the policy to gain permissions
- D) The user gets permissions only if their account has MFA enabled

Answer: B Reasoning: IAM users inherit permissions from groups through **policy attachment**. The policy remains attached to the group — it is not copied or converted into an inline policy. Permissions take effect immediately; no explicit acceptance is needed, and MFA is irrelevant here.

Q10. An IAM policy has an **Allow** for **iam:PassRole**. Why is this permission often considered highly sensitive?

- A) It lets a user create other IAM users
- B) It lets a user assign a role to an AWS service, potentially granting that service elevated privileges
- C) It gives full IAM administrator access
- D) It allows a user to assume any role in the account

Answer: B Reasoning: **iam:PassRole** allows a user to pass an IAM role to an AWS service (e.g., EC2, Lambda). If an attacker has **iam:PassRole** and can launch EC2 or Lambda, they could pass an **admin-level role** to that service and use it to escalate

privileges — even without having those permissions themselves. This is a classic privilege escalation vector.

Q11. A policy uses "`Principal`": "*" with a `Condition` requiring `aws:SourceVpc`. Where can this policy be used?

- A) As an IAM identity policy
- B) As a resource-based policy (e.g., S3 bucket policy)
- C) As a permissions boundary
- D) As a service control policy (SCP)

Answer: B Reasoning: The `Principal` element is **only valid** in **resource-based policies** (S3, SQS, SNS, KMS key policies, etc.) and trust policies. Identity-based policies (including permissions boundaries and SCPs) do **not** support the `Principal` element.

Q12. An IAM policy uses "`NotAction`": ["iam:*", "organizations:*"] with "`Effect`": "Allow". What does this grant?

- A) Deny access to IAM and Organizations actions
- B) Allow all actions except IAM and Organizations actions
- C) Allow only IAM and Organizations actions
- D) Deny all actions

Answer: B Reasoning: `NotAction` with `Effect: Allow` means "Allow every action **except** the listed ones." This is commonly used in SCP guardrails to prevent privilege escalation by blocking IAM changes while allowing all other services.

Q13. Which combination of policy types can result in an effective permission for an IAM role in an AWS Organization?

- A) SCP + Identity Policy only
- B) SCP + Identity Policy + Permissions Boundary + Resource Policy
- C) Identity Policy + Resource Policy only
- D) SCP + Resource Policy only

Answer: B Reasoning: Effective permissions for an IAM principal in an organization are determined by the intersection of: **SCP** (org-level guardrail), **Permissions Boundary** (max permissions), **Identity Policy** (what's granted), and for resource access, a **Resource Policy** may also be required (especially cross-account). All layers must allow the action.

Q14. A user can call `sts:GetCallerIdentity` without any IAM policies attached. Why?

- A) It's a public AWS API that requires no authentication
- B) `sts:GetCallerIdentity` is always allowed regardless of IAM policies
- C) The root account implicitly grants it to all users
- D) It requires only MFA, not IAM policies

Answer: B Reasoning: `sts:GetCallerIdentity` is special — it is **always permitted** for any authenticated caller, regardless of attached policies. AWS intentionally exempts it from policy evaluation. It returns the account ID, user ID, and ARN of the calling identity.

Q15. An IAM policy grants `s3:*` on `arn:aws:s3:::my-bucket`. A user tries to list objects using `s3:ListObjectsV2` on `arn:aws:s3:::my-bucket/*`. Does this work?

- A) Yes — `s3:*` covers all S3 actions on any resource
- B) No — S3 bucket-level vs. object-level ARNs matter; the action is on the wrong ARN
- C) Yes — wildcards in actions cover both bucket and object ARNs
- D) No — `s3:ListObjectsV2` is not part of `s3:*`

Answer: B Reasoning: S3 has a subtle distinction: **bucket-level actions** (like `s3:ListBucket`) apply to `arn:aws:s3:::my-bucket`, while **object-level actions** (like `s3:GetObject`, `s3:PutObject`) apply to `arn:aws:s3:::my-bucket/*`. `s3:ListObjectsV2` is a **bucket-level action** and requires the bucket ARN, not the object ARN. The policy targets the bucket ARN, so `ListObjectsV2` IS covered — but `GetObject` would require the object ARN.

SECTION 2: STS, Roles & Federation (Q16–Q35)

Q16. When a user assumes an IAM role using `sts:AssumeRole`, what happens to their original IAM user permissions?

- A) They are combined with the role's permissions
- B) They are suspended; only the role's permissions apply
- C) The most permissive of the two sets applies
- D) They remain active as a fallback

Answer: B Reasoning: When you assume a role, you **give up your original identity's permissions** and operate solely under the assumed role's permissions. There is no combining or fallback. This is why role chaining can reduce permissions — you're always starting fresh with the role.

Q17. An IAM role has a maximum session duration of 1 hour. A user assumes it with `DurationSeconds=7200` (2 hours). What happens?

- A) The session lasts 2 hours — the parameter overrides the role setting
- B) The API call fails with a validation error
- C) The session defaults to 1 hour without error
- D) The session lasts 12 hours — the account maximum applies

Answer: B Reasoning: If you request a session duration longer than the role's configured **maximum session duration**, the `AssumeRole` API call **returns an error**. You must request a duration less than or equal to the role's maximum. The default maximum is 1 hour; it can be set up to 12 hours.

Q18. Which temporary credential token component is used to make signed API calls?

- A) `AccessKeyId` only
- B) `SecretAccessKey` only
- C) `AccessKeyId` + `SecretAccessKey` + `SessionToken`
- D) `SessionToken` only

Answer: C Reasoning: STS temporary credentials have **three components**: `AccessKeyId`, `SecretAccessKey`, and a `SessionToken`. All three must be used together when making API calls. If you omit the session token, the call fails with an `InvalidClientTokenId` error — a common gotcha.

Q19. A Lambda function in Account A needs to access DynamoDB in Account B. What is the correct approach?

- A) Create an IAM user in Account B and embed credentials in the Lambda environment variables
- B) Create a role in Account B with a trust policy for Account A's Lambda, and grant the Lambda role in Account A `sts:AssumeRole` for that role
- C) Use AWS Config to share DynamoDB access across accounts
- D) Enable cross-account access via the DynamoDB resource policy

Answer: B Reasoning: The correct cross-account pattern is: (1) Create a role in **Account B** with a trust policy allowing Account A's Lambda execution role to assume it, (2) Grant the Lambda's execution role `sts:AssumeRole` for the Account B role. DynamoDB does **not** support resource-based policies for cross-account access — unlike S3 or SQS. Hardcoding credentials is a security anti-pattern.

Q20. What is the purpose of `ExternalId` in an STS `AssumeRole` call?

- A) To encrypt the session token
- B) To prevent the "confused deputy" problem in cross-account access
- C) To set the maximum session duration
- D) To scope down permissions within the session

Answer: B Reasoning: `ExternalId` is a unique identifier that a third party provides when assuming a role. It mitigates the **confused deputy attack**, where a malicious actor tricks a trusted service into assuming a role on their behalf. By requiring a secret `ExternalId` known only to the legitimate caller, AWS prevents unauthorized role assumptions.

Q21. An IAM user in Account A assumes a role in Account B. The temporary credentials expire. The user tries to use the expired credentials. What error is returned?

- A) `AccessDeniedException`
- B) `TokenRefreshRequired`
- C) `ExpiredTokenException`
- D) `InvalidUserException`

Answer: C Reasoning: When temporary credentials from STS expire, subsequent API calls return `ExpiredTokenException`. The application must call `AssumeRole` again to obtain fresh credentials. Temporary credentials cannot be refreshed — they must be regenerated.

Q22. Web Identity Federation using Amazon Cognito vs. using `AssumeRoleWithWebIdentity` directly — what is the key advantage of Cognito?

- A) Cognito supports more OIDC providers
- B) Cognito provides user pools, token vending, and manages the `AssumeRoleWithWebIdentity` call internally, abstracting credential management
- C) Cognito bypasses IAM policies
- D) Cognito allows longer session durations

Answer: B Reasoning: AWS recommends Cognito over direct `AssumeRoleWithWebIdentity` because Cognito **manages the entire federation flow** — it handles token exchange, refreshes, and credential vending. It also supports **user pools** (authentication) and **identity pools** (authorization). Direct `AssumeRoleWithWebIdentity` puts all token management on the developer.

Q23. A company uses SAML 2.0 federation with IAM. After a user authenticates with the corporate IdP, the IdP sends a SAML assertion to which AWS endpoint?

- A) `https://iam.amazonaws.com/saml`
- B) `https://signin.aws.amazon.com/saml`
- C) `https://sts.amazonaws.com/saml`
- D) `https://console.aws.amazon.com/federation`

Answer: B Reasoning: The SAML assertion is POSTed to `https://signin.aws.amazon.com/saml` for **console access**. For **programmatic**

access, the IdP calls `sts:AssumeRoleWithSAML`. These are different flows. The console endpoint is the common trick answer — many confuse it with the STS endpoint.

Q24. An IAM role's trust policy allows `"Service": "ec2.amazonaws.com"`. A Lambda function tries to assume this role. What happens?

- A) Lambda can assume it — service trusts are interchangeable
- B) Lambda cannot assume it — the trust policy does not allow Lambda
- C) Lambda can assume it after MFA verification
- D) Lambda can assume it if it has `sts:AssumeRole` permission

Answer: B Reasoning: The trust policy strictly defines **who can assume the role**. `ec2.amazonaws.com` is an EC2-specific service principal. Lambda uses `lambda.amazonaws.com`. A Lambda function **cannot** assume a role trusted only for EC2, regardless of other permissions.

Q25. What is **role chaining** and what is its limitation?

- A) Attaching multiple policies to a role; limited to 10 policies
- B) Assuming a role that then assumes another role; the maximum session duration is capped at 1 hour
- C) Using a role across multiple accounts; limited to 5 accounts
- D) Nesting roles inside groups; limited to 3 levels

Answer: B Reasoning: Role chaining occurs when you use temporary credentials from one role to assume another role. AWS limits chained role sessions to a **maximum of 1 hour**, regardless of the role's configured maximum session duration. This is a significant gotcha for automation pipelines.

Q26. A user assumes a role with `--policy` (inline session policy) in addition to the role's permission policy. How does the inline session policy affect permissions?

- A) It adds permissions to the role's policy
- B) It replaces the role's policy entirely
- C) It further restricts permissions to the intersection of the role policy and the session policy

- D) It has no effect if the role already has a policy

Answer: C Reasoning: A **session policy** (passed inline during `AssumeRole`) **cannot grant more permissions** than the role's permissions policy. It can only **restrict** the effective permissions to the intersection of the role policy and the session policy. This is used to scope down a role for a specific task.

Q27. Which STS action should be used when the caller is already authenticated via AWS (not federated)?

- A) `AssumeRoleWithWebIdentity`
- B) `AssumeRoleWithSAML`
- C) `AssumeRole`
- D) `GetFederationToken`

Answer: C Reasoning: `AssumeRole` is for AWS-authenticated callers (IAM users, roles, services). `AssumeRoleWithWebIdentity` is for OAuth/OIDC tokens (Google, Facebook, Cognito). `AssumeRoleWithSAML` is for SAML 2.0 assertions. `GetFederationToken` is for federating existing IAM user credentials (now largely superseded).

Q28. An application uses `sts:GetFederationToken` to generate temporary credentials for end users. The calling IAM user has `s3:*` permissions. What is the maximum permission a federated user can have?

- A) `s3:*` — inherits all caller permissions
- B) Only what is explicitly granted in the federation token policy, up to the caller's permissions
- C) Full account access — federation tokens bypass IAM
- D) Read-only access — federation tokens are always scoped down

Answer: B Reasoning: `GetFederationToken` creates credentials scoped to the **intersection** of (1) the calling IAM user's policies and (2) the policy passed in the `GetFederationToken` call. A federated user **cannot exceed** the caller's permissions. If no policy is passed, the federated user gets no permissions.

Q29. A company requires that no IAM role can be assumed without MFA. How is this enforced?

- A) Enable MFA at the AWS Organizations level via SCPs
- B) Add a Condition `"aws:MultiFactorAuthPresent": "true"` to the role's **trust policy**
- C) Add `"aws:MultiFactorAuthPresent": "true"` to each role's permissions policy
- D) Enable AWS Config MFA rule

Answer: B Reasoning: To require MFA for role assumption, the condition must be in the **trust policy** (`AssumeRolePolicyDocument`), not the permissions policy. The trust policy controls who can assume the role. Adding the MFA condition in the permissions policy would only restrict actions after assumption, not the assumption itself.

Q30. After assuming a role, can an IAM user still use their original IAM user credentials simultaneously?

- A) No — role assumption revokes the original credentials
- B) Yes — credentials are independent; the original IAM user credentials remain valid
- C) No — AWS enforces single-session per user
- D) Yes — but only for read-only operations

Answer: B Reasoning: Assuming a role does **not** invalidate or suspend original IAM user credentials. The user simply has **two sets of active credentials**: the original IAM user credentials and the temporary role credentials. They can use either, with the respective permissions of each identity.

Q31. An STS temporary credential's `AccessKeyId` always starts with which prefix?

- A) `AKIA`
- B) `ASIA`
- C) `AROA`
- D) `AGPA`

Answer: B Reasoning: AWS uses key prefixes to identify credential types: **AKIA** = IAM user long-term access key; **ASIA** = STS temporary credentials; **AROA** = IAM role ID; **AGPA** = IAM group ID; **AIDA** = IAM user ID. Recognizing **ASIA** as STS credentials is useful for security audits.

Q32. A Lambda function's execution role has **s3:GetObject** on a specific bucket. A developer adds an inline session policy with **s3:*** when invoking Lambda. What can Lambda do?

- A) **s3:*** — session policy adds permissions
- B) Only **s3:GetObject** — session policy cannot exceed role permissions
- C) **s3:*** — Lambda ignores session policies
- D) Nothing — conflicting policies result in Deny

Answer: B Reasoning: Session policies are **restrictive only** — they can narrow permissions but never expand them beyond the role's permission policy. If the execution role only has **s3:GetObject**, passing **s3:*** as a session policy still results in effective permissions of only **s3:GetObject**.

Q33. Which IAM feature allows you to grant an entity only the permissions needed, without modifying the entity's existing permission policies?

- A) Permission Boundaries
- B) Service Control Policies
- C) Session Policies
- D) Trust Policies

Answer: C Reasoning: Session policies are passed at the time of **AssumeRole** and restrict permissions for that specific session without altering the role's policy. Permission boundaries set maximum permissions persistently. SCPs are org-level. Trust policies control assumption. Session policies are temporary, scoped restrictions.

Q34. An OIDC provider is configured in IAM for GitHub Actions. The trust policy uses **token.actions.githubusercontent.com** as the federated principal. A

Condition with `StringEquals` checks `sub` for a specific repo. What does this prevent?

- A) Replay attacks using expired tokens
- B) Any GitHub Actions workflow from assuming the role; only specific repos can
- C) MITM attacks on the OIDC token
- D) Users from manually calling `AssumeRoleWithWebIdentity`

Answer: B Reasoning: The `sub` (subject) claim in the OIDC token identifies the specific repository, branch, or environment. By using `StringEquals` on `sub`, you restrict assumption to a **specific repository's workflows**. Without this condition, **any** GitHub Actions workflow in any public repo could assume the role — a critical security misconfiguration.

Q35. When using `AssumeRoleWithWebIdentity`, which claim from the JWT token maps to the `aws:userId` variable?

- A) `aud` (audience)
- B) `sub` (subject)
- C) `iss` (issuer)
- D) `exp` (expiration)

Answer: B Reasoning: In `AssumeRoleWithWebIdentity`, the `sub` claim from the OIDC/JWT token maps to the `aws:userId` variable in IAM policy conditions. The `aud` maps to `token.actions.githubusercontent.com` (or the configured audience). Knowing this is key for writing fine-grained OIDC-based trust policies.

SECTION 3: Organizations, SCPs & Guardrails (Q36–Q50)

Q36. A Service Control Policy (SCP) with `Allow: s3:*` is attached to an OU. An IAM user in that OU has an identity policy with `Allow: ec2:*`. Can the user access EC2?

- A) Yes — identity policies are independent of SCPs
- B) No — the SCP restricts the maximum permissions to S3 only
- C) Yes — EC2 is allowed by default in all accounts

- D) No — SCPs override identity policies entirely

Answer: B Reasoning: SCPs define the **maximum available permissions** for accounts in an OU. The effective permission = intersection of SCP and identity policy. The SCP only allows `s3:*`, so `ec2:*` is **not in the SCP's allow set**, making EC2 access impossible regardless of the identity policy. SCPs are a **guardrail**, not a grant.

Q37. Do SCPs apply to the **management account** of an AWS Organization?

- A) Yes — SCPs apply to all accounts including the management account
- B) No — SCPs do not affect the management account
- C) Only if explicitly enabled via AWS Config
- D) Only for service-linked roles in the management account

Answer: B Reasoning: This is a critical and commonly missed fact: **SCPs do not apply to the management (master) account**. They only affect member accounts in the organization. This means IAM users and roles in the management account are **not restricted** by SCPs — only by their IAM policies.

Q38. An SCP is attached to the **Root** of an organization with **Deny: `ec2:TerminateInstances`**. A member account administrator attaches an IAM policy with **Allow: `ec2:TerminateInstances`** to a user. What can the user do?

- A) Terminate instances — IAM allows it
- B) Cannot terminate instances — SCP Deny overrides the IAM Allow
- C) Can terminate instances only in the same region as the SCP
- D) Can terminate instances after assuming a role with admin access

Answer: B Reasoning: An explicit **Deny** in an SCP **always overrides** any IAM Allow. SCPs act as a guardrail at the organization level. Even account administrators cannot override an SCP Deny — they would need to remove the SCP at the organization level, which requires management account access.

Q39. If an AWS account has **no SCPs attached** (other than the default **`FullAWSAccess`**), what is the effect?

- A) All actions are denied by default — SCPs must explicitly allow

- B) All actions permitted by identity policies are allowed — the full access SCP is the default
- C) Only AWS-managed services are accessible
- D) The account operates as a standalone account without organizational guardrails

Answer: B Reasoning: By default, AWS attaches a **FullAWSAccess** SCP to all OUs and accounts, which allows all actions. This means SCPs don't restrict anything unless you explicitly **add Deny statements** or **replace** the **FullAWSAccess** policy with a more restrictive one. Organizations use a **deny-list strategy** (start with full access, add denies) or **allow-list strategy** (remove full access, add specific allows).

Q40. A company uses the **allow-list SCP strategy** — they remove the **FullAWSAccess** SCP and attach a custom SCP allowing only specific services. An IAM admin in a member account has **AdministratorAccess** IAM policy. What can they do?

- A) Everything — **AdministratorAccess** overrides SCPs
- B) Only actions explicitly listed in the SCP — **AdministratorAccess** is still bounded by the SCP
- C) Everything except billing and organizations
- D) Nothing — removing **FullAWSAccess** revokes all permissions

Answer: B Reasoning: SCPs are an upper bound. Even **AdministratorAccess** (which grants ***:***) cannot exceed what the SCP allows. In the allow-list strategy, **only services explicitly listed in the SCP** are accessible, regardless of the IAM policy. This is how organizations implement strict service restriction.

Q41. Can SCPs be used to grant access to AWS resources?

- A) Yes — SCPs can explicitly grant permissions
- B) No — SCPs only define maximum permissions; actual grants must come from IAM policies
- C) Yes — but only for root-level actions
- D) No — SCPs can only deny; they cannot allow or deny specific resources

Answer: B Reasoning: SCPs are **not grants** — they define the maximum available permissions (guardrails). Even if an SCP allows an action, an IAM identity still needs an IAM policy that grants that action. SCPs alone do **not** give anyone access; they only limit what IAM can grant.

Q42. A **Deny** SCP has a **Condition** that checks **aws:RequestedRegion**. What is this used for?

- A) Prevent users from creating resources in specific AWS regions
- B) Allow access only from specific IP ranges
- C) Restrict S3 bucket regions
- D) Enforce data residency by allowing requests only in specific regions

Answer: D (also A is correct, but D is the best answer) **Reasoning:** Using **aws:RequestedRegion** in an SCP allows you to **restrict which AWS regions** member accounts can operate in. For example, a European company can deny all regions except **eu-west-1** and **eu-central-1** to enforce data residency requirements. This is a common enterprise compliance pattern.

Q43. Which of the following **cannot** be restricted by an SCP?

- A) IAM root user actions in member accounts
- B) Service-linked roles
- C) IAM user access key creation
- D) S3 bucket creation

Answer: B Reasoning: Service-linked roles (like those created by AWS Config, CloudTrail, or other services) **cannot be restricted by SCPs** because they are tied to specific AWS service behaviors. Trying to restrict their actions via SCPs can break AWS service functionality. Root user actions in member accounts **can** be restricted by SCPs (unlike the management account root).

Q44. An organization has 3 levels of OUs: Root → OU1 → OU2. SCPs at each level: Root has **Deny ec2:TerminateInstances**, OU1 has **Allow ec2:***, OU2 has **Allow ec2:TerminateInstances**. An account in OU2 has an IAM user with **Allow ec2:TerminateInstances**. Can the user terminate instances?

- A) Yes — the most specific OU level wins
- B) No — the Root SCP Deny propagates down and cannot be overridden
- C) Yes — OU2's Allow overrides Root's Deny
- D) No — conflicting SCPs result in implicit Deny

Answer: B Reasoning: SCPs are **inherited** through the OU hierarchy, and an explicit **Deny** at any level propagates down to all child OUs and accounts. A child OU or account **cannot override** a Deny from a parent SCP. The Root-level Deny blocks **ec2:TerminateInstances** for all accounts, period.

Q45. What is the maximum number of SCPs that can be attached to a single AWS account directly?

- A) 1
- B) 5
- C) 10
- D) 25 (within overall limits)

Answer: B Reasoning: AWS allows a **maximum of 5 SCPs** attached to any single entity (root, OU, or account). If you need more complex guardrails, you must consolidate them into fewer policies. This limit often forces architects to think carefully about SCP design.

SECTION 4: Resource-Based Policies & Cross-Account (Q46–Q60)

Q46. An S3 bucket in Account A has a bucket policy allowing **s3:GetObject** for Account B's IAM role ARN. Account B's role has no IAM policy for S3. Can the role access the bucket?

- A) No — the role must have a matching IAM policy in Account B
- B) Yes — for same-account access, only one policy is needed; for cross-account, both are required
- C) Yes — the bucket policy alone is sufficient for cross-account access
- D) No — cross-account access always requires explicit trust relationships

Answer: B — the trick is in the framing. **Answer: B** explains it but let's restate: **for cross-account S3 access**, both the **bucket policy** (in Account A) AND an **IAM policy** in Account B must allow the action.

Reasoning: This is a critical cross-account distinction. For **same-account** access, either a bucket policy OR an IAM policy is sufficient. For **cross-account** access, **both** are required: (1) the resource policy must allow the cross-account principal, AND (2) the cross-account identity must have an IAM policy allowing the action.

Q47. A KMS key policy does NOT include the account's root ARN in the **Principal**. An IAM policy in the same account grants a user **kms:Decrypt**. Can the user decrypt using the key?

- A) Yes — IAM policies always take precedence over KMS key policies
- B) No — KMS key policies are the **sole access control** for KMS; without access in the key policy, IAM policies are insufficient
- C) Yes — same-account access auto-delegates to IAM
- D) No — the user needs both the IAM policy and the key policy to allow them

Answer: B Reasoning: KMS key policies are unique — **unlike S3**, KMS requires explicit access in the **key policy** for IAM policies to take effect. If the key policy does not include the account root or the specific IAM entity, no IAM policy can grant access to the key. The key policy is the **gatekeeper** for all KMS access. This is the most commonly missed KMS/IAM nuance.

Q48. An S3 bucket policy uses **"Principal": {"AWS": "arn:aws:iam::123456789012:root"}**. What does this allow?

- A) Only the root user of account 123456789012
- B) All IAM identities in account 123456789012, delegating access control to IAM
- C) Only the management account of the organization
- D) All AWS accounts in the same organization

Answer: B Reasoning: **arn:aws:iam::ACCOUNT_ID:root** in a resource policy represents the **entire AWS account**, not just the root user. It delegates access control to IAM policies in that account. Any IAM user or role in account 123456789012 with an appropriate IAM policy can access the resource. This is the **cross-account delegation** pattern.

Q49. Which AWS services support **resource-based policies**? (Choose the odd one out — the one that does NOT support resource-based policies)

- A) S3
- B) SQS
- C) DynamoDB
- D) Lambda

Answer: C Reasoning: **DynamoDB does NOT support resource-based policies** (as of the SAA-C02 exam scope). Cross-account DynamoDB access requires role-based access (assuming a role in the target account). S3, SQS, SNS, Lambda, ECR, Secrets Manager, and KMS all support resource-based policies.

Q50. A Lambda function's resource-based policy allows **Account B** to invoke it. Account B's user has no IAM policy. Can the user invoke Lambda?

- A) Yes — Lambda resource policies are sufficient for cross-account invocation
- B) No — the user still needs **lambda:InvokeFunction** in their IAM policy
- C) Yes — Lambda auto-grants invoke to any principal in the trust list
- D) No — Lambda does not support resource-based policies

Answer: B Reasoning: Cross-account access to Lambda (and most services with resource policies) requires **both** the resource policy to allow the principal AND the principal's IAM identity policy to allow the action. This mirrors the S3 cross-account pattern. (Note: For S3 access from the same account, one policy suffices — but cross-account always needs both.)

Q51. An IAM role in Account A accesses an S3 bucket in Account B. The bucket is owned by Account B but contains objects uploaded by Account A. Who owns those objects?

- A) Account B — the bucket owner owns all objects
- B) Account A — the uploader owns objects by default
- C) AWS — S3 objects have no IAM owner
- D) Shared — both accounts have equal ownership

Answer: B Reasoning: By default, the **IAM entity that uploads** an S3 object owns it, not the bucket owner. This is the **cross-account object ownership problem**. Even if Account B owns the bucket, Account B may not be able to read or delete objects uploaded by Account A. The fix is to use **S3 Object Ownership** (**BucketOwnerEnforced**) or require ACLs that grant bucket-owner-full-control.

Q52. A company enables **S3 Object Ownership** with **BucketOwnerEnforced**. What happens to bucket ACLs?

- A) ACLs are still supported but bucket owner controls all objects
- B) All ACLs are disabled; the bucket owner automatically owns all objects
- C) ACLs are replaced by bucket policies only
- D) Only public ACLs are disabled; private ACLs still work

Answer: B Reasoning: **BucketOwnerEnforced disables all ACLs** (both bucket and object ACLs) and makes the bucket owner the automatic owner of all objects, regardless of who uploads them. This is the recommended setting as of 2023 — AWS even made it the default for new buckets. Bucket policies become the sole access control mechanism.

Q53. A VPC Endpoint policy restricts **s3:GetObject** to a specific bucket. An IAM role within the VPC has **s3:*** on all buckets. Traffic goes through the VPC endpoint. What can the role access?

- A) All S3 buckets — IAM policy is more permissive
- B) Only the specific bucket defined in the VPC endpoint policy
- C) No buckets — conflicting policies block all access
- D) All buckets — VPC endpoint policies only apply to public traffic

Answer: B Reasoning: When accessing S3 through a **VPC endpoint**, all three layers apply: IAM policy, resource policy, and **VPC endpoint policy**. The effective permission is the intersection of all three. The VPC endpoint policy restricts access to one bucket, so even though the IAM role has **s3:***, only that specific bucket is accessible through the endpoint.

Q54. A bucket policy uses `Condition: {"StringNotEquals": {"aws:sourceVpc": "vpc-12345"}}` with `Effect: Deny`. What is the purpose?

- A) Allow access only from vpc-12345
- B) Deny access from vpc-12345
- C) Block all public internet access and allow only vpc-12345
- D) Allow access only from the VPC's CIDR range

Answer: C Reasoning: This is a **private S3 bucket pattern**. `StringNotEquals` with `Deny` means: "Deny if the source VPC is NOT vpc-12345." Effectively, all requests NOT from vpc-12345 are denied, making the bucket accessible **only from that VPC**. This enforces private-only access without a bucket policy that explicitly lists every allowed IP.

Q55. An SNS topic in Account A needs to publish to an SQS queue in Account B. What is required?

- A) SNS resource policy in Account A allowing Account B's SQS
- B) SQS resource policy in Account B allowing Account A's SNS topic ARN
- C) An IAM role in Account B that SNS can assume
- D) AWS PrivateLink connection between accounts

Answer: B Reasoning: For SNS → SQS cross-account delivery, the **SQS queue's resource policy** must allow SNS to send messages (`sqs:SendMessage`) from the SNS topic ARN in Account A. SNS does not assume roles to deliver messages to SQS; it uses the resource policy on the queue.

SECTION 5: Permissions Boundaries (Q56–Q65)

Q56. A permissions boundary is attached to an IAM user. The boundary allows `ec2:*`. The user's identity policy allows `ec2:*` and `s3:*`. An S3 bucket policy explicitly allows the user `s3:GetObject`. What can the user access?

- A) `ec2:*` and `s3:GetObject`
- B) Only `ec2:*`
- C) `ec2:*`, `s3:*`

- D) `ec2:*` and all S3 actions allowed by the bucket policy

Answer: B Reasoning: Permissions boundaries restrict identity-based access. The boundary allows `ec2:*` but not `s3:*`. For **S3 access**, even though the bucket policy allows it and the identity policy allows it, the **permissions boundary blocks it** because `s3:*` is not in the boundary. All three (identity policy, boundary, resource policy for cross-account) must allow the action.

Q57. A developer has `iam:CreateRole` and `iam:AttachRolePolicy` permissions but **no permissions boundary** on their IAM user. They create a new IAM role. What prevents them from escalating privileges?

- A) SCPs
- B) Nothing — they can create an admin role and assign it to themselves
- C) IAM Conditions
- D) AWS Config

Answer: B Reasoning: Without a **permissions boundary** constraint on `iam:CreateRole`, a user with `iam:CreateRole` and `iam:AttachRolePolicy` can create a role with `AdministratorAccess` and then assume it, **fully escalating privileges**. This is why granting IAM management permissions without permissions boundaries is dangerous. The best practice is to require that created roles inherit a permissions boundary.

Q58. How can an administrator prevent developers from creating IAM entities with permissions beyond their own level?

- A) Use IAM Access Analyzer
- B) Attach a permissions boundary to developer users and require a Condition on `iam:CreateRole` that the new role also has the boundary
- C) Use SCPs to restrict IAM actions
- D) Enable MFA for all IAM actions

Answer: B Reasoning: The pattern uses a Condition like

`"iam:PermissionsBoundary":`

`"arn:aws:iam::ACCOUNT:policy/DeveloperBoundary"` in the policy governing `iam:CreateRole` and `iam:CreateUser`. This forces any IAM entity a developer

creates to have the same (or more restrictive) permissions boundary, preventing privilege escalation.

Q59. A permissions boundary is attached to a role. The boundary grants `iam:CreateUser`. The role's identity policy does NOT grant `iam:CreateUser`. Can the role create IAM users?

- A) Yes — the boundary grants it
- B) No — the boundary defines the maximum; actual access requires the identity policy to also grant it
- C) Yes — IAM admin actions bypass the intersection rule
- D) No — IAM actions always require explicit Deny to block

Answer: B Reasoning: A permissions boundary is the **ceiling**, not a grant. The actual permission must exist in the **identity policy** (or a resource policy). If the identity policy doesn't have `iam:CreateUser`, the role cannot create users — even if the boundary permits it. Both must allow.

Q60. Permissions boundaries can be attached to which IAM entity types?

- A) Users, groups, and roles
- B) Users and roles only
- C) Roles only
- D) Users, roles, and AWS services

Answer: B Reasoning: Permissions boundaries can only be applied to **IAM users and IAM roles**. They **cannot** be applied to IAM groups. This is a key distinction — attaching a boundary to a group is not possible; you must attach it to individual users or roles.

SECTION 6: IAM Policies — Advanced (Q61–Q75)

Q61. What is the difference between `aws:PrincipalArn` and `aws:SourceArn` in policy conditions?

- A) Both refer to the calling IAM identity; they're interchangeable

- B) `aws:PrincipalArn` is the IAM identity making the request; `aws:SourceArn` is the AWS service calling on behalf of another resource
- C) `aws:PrincipalArn` is for cross-account only; `aws:SourceArn` is for same-account
- D) `aws:SourceArn` is deprecated in favor of `aws:PrincipalArn`

Answer: B Reasoning: `aws:PrincipalArn` identifies the **IAM user/role** making the API call. `aws:SourceArn` is used in **service-to-service** scenarios to identify the originating AWS resource (e.g., an S3 bucket that triggers a Lambda). `aws:SourceArn` is used in resource-based policies to limit which specific resource can trigger an action — mitigating confused deputy attacks.

Q62. A policy uses `"Condition": {"DateLessThan": {"aws:CurrentTime": "2024-12-31T00:00:00Z"}}` with `Effect: Allow`. What happens after December 31, 2024?

- A) The policy still allows — conditions don't expire policies
- B) The Allow no longer applies — the condition fails, resulting in implicit Deny
- C) The policy auto-deletes after the date
- D) Access becomes read-only

Answer: B Reasoning: A time-based condition on an `Allow` means the Allow only applies when the condition is true (before the date). After the specified date, the condition fails, the Allow statement does not apply, and the default **implicit Deny** takes effect. This pattern is used for temporary access grants.

Q63. What is the `aws:RequestedRegion` condition key used for, and where does it NOT work?

- A) Restricts API calls to specific regions; works for all AWS services
- B) Restricts API calls to specific regions; does NOT work for global services like IAM, Route 53, CloudFront (which use us-east-1 as their endpoint)
- C) Restricts S3 bucket regions only
- D) Works only in SCPs, not identity policies

Answer: B Reasoning: `aws:RequestedRegion` restricts which AWS regions an API call targets. However, **global services** (IAM, STS, Route 53, CloudFront, AWS

Organizations) don't have regional endpoints — they all route through `us-east-1`. Adding a region restriction may inadvertently block these services unless `us-east-1` is in the allowed list.

Q64. An IAM policy uses `"StringEquals": {"s3:prefix": ["finance/", "hr/"]}` in a Condition. What does this enable?

- A) Restrict uploads to specific S3 prefixes
- B) Restrict S3 `ListBucket` operations to specific prefixes (folder-level access)
- C) Encrypt only objects with specific prefixes
- D) Block downloads from specific prefixes

Answer: B Reasoning: The `s3:prefix` condition key works with `s3:ListBucket` to simulate **folder-level access** in S3. Users can list objects only under the specified prefixes. This, combined with an object-level policy using `s3:prefix` in the resource ARN, enables S3 "home folder" patterns.

Q65. What is the `aws:TagKeys` condition key used for?

- A) Require specific tag keys on IAM roles
- B) Restrict which tag keys can be applied when creating or tagging resources
- C) Enforce tag-based access control for S3 objects
- D) Restrict access to resources based on their existing tags

Answer: B Reasoning: `aws:TagKeys` is used in conditions to **control which tag keys** are applied in `CreateResource` or `TagResource` API calls. For example, you can prevent users from adding a `Environment:Production` tag to prevent unauthorized modification of production resources. It's a preventive control, not a resource-filtering control.

Q66. What is **Attribute-Based Access Control (ABAC)** in AWS IAM?

- A) Controlling access based on the source IP address
- B) Using tags on both IAM principals and resources to determine access
- C) Using AWS Organizations attributes to control access
- D) Role-based access using department attributes

Answer: B Reasoning: ABAC in AWS uses **resource tags** and **principal tags** together in policy conditions. For example: `"StringEquals": {"aws:ResourceTag/Project": "${aws:PrincipalTag/Project}"}` — allows access only if the resource's **Project** tag matches the user's **Project** tag. This scales well because you don't need to update policies when adding new resources; you just tag them appropriately.

Q67. An IAM policy has `"Effect": "Allow"` and `"Resource": "*" . An SCP also has "Effect": "Allow" and "Resource": "*" . An IAM user tries to access a service. What determines if access is granted?`

- A) Both are Allow wildcards, so all access is granted
- B) The IAM policy must list the specific action; wildcards in Resource don't grant actions
- C) The SCP must list the specific action; wildcards in Resource don't grant actions
- D) Both must explicitly list the action for access to be granted

Answer: D (but B and C are also important notes) **Reasoning:** `"Resource": "*" . means the policy applies to all resources, but the Action element must still explicitly list the permitted actions. A policy with "Resource": "*" and no Action (or an empty one) grants nothing. Both the SCP and IAM policy must include explicit Action grants for access.`

Q68. What happens when a policy has no `Condition` block at all?

- A) It applies only during business hours by default
- B) It applies unconditionally — all requests matching the Action/Resource are governed by the Effect
- C) It defaults to requiring MFA
- D) It only applies to API calls, not console access

Answer: B Reasoning: A policy without a `Condition` block applies **unconditionally** to any request matching its Action and Resource. Conditions are optional refinements. The absence of a Condition means the policy applies regardless of request context (IP, time, MFA status, etc.).

Q69. What is the maximum size of an inline policy document attached to an IAM user?

- A) 2 KB
- B) 10 KB
- C) 6,144 characters
- D) 20,480 characters

Answer: C — but let's clarify: Reasoning: The character limit for inline policies is **2,048 characters per user, 5,120 per group, 10,240 per role** for the sum of all inline policies. Individual customer-managed policies can be up to **6,144 characters**. The key point: inline policies have tighter limits than managed policies, and you're limited to the total allowed per entity type.

Q70. What is the difference between `iam:PassRole` and `sts:AssumeRole`?

- A) They are equivalent — both allow role assumption
- B) `iam:PassRole` allows assigning a role to an AWS service; `sts:AssumeRole` allows a principal to assume a role directly
- C) `sts:AssumeRole` is for cross-account only; `iam:PassRole` is for same-account
- D) `iam:PassRole` requires MFA; `sts:AssumeRole` does not

Answer: B Reasoning: `iam:PassRole` is needed when **assigning a role to an AWS service** during resource creation (e.g., passing an EC2 instance profile, a Lambda execution role, an ECS task role). `sts:AssumeRole` is what a principal calls to **actively assume** a role and receive temporary credentials. They serve completely different purposes.

Q71. An IAM policy JSON has a typo — the `"Effect"` field is misspelled as `"Effekt"`. What happens?

- A) AWS auto-corrects the typo
- B) The policy fails validation and cannot be saved
- C) The policy is saved but has no effect (treated as invalid)
- D) The statement is treated as an implicit Deny

Answer: B Reasoning: IAM policy JSON is validated at **creation time**. A misspelled or invalid key like `"Effekt"` will cause the API call to fail with a `MalformedPolicyDocumentException`. AWS validates policy syntax strictly — you cannot save an invalid policy document.

Q72. An IAM policy uses `"NotPrincipal"` with `"Effect": "Deny"`. What is the risk?

- A) It denies all principals including the root user
- B) It is functionally equivalent to `Principal`
- C) It can accidentally lock out the account if not combined with a Deny exception for the root
- D) It only works with resource policies, not identity policies

Answer: C Reasoning: `NotPrincipal` with `Effect: Deny` means "deny everyone EXCEPT the listed principals." If you don't include the **root account** or admin roles in the exception list, you risk **locking out all administrators**. AWS explicitly warns about this pattern — `NotPrincipal` with Deny should always include emergency access principals.

Q73. What is the `aws:ViaAWSService` condition key used for?

- A) Restrict which AWS services can call the API
- B) Differentiate between direct user API calls and service-made calls on behalf of the user
- C) Restrict access from outside VPC endpoints
- D) Identify federated users

Answer: B Reasoning: `aws:ViaAWSService` is `true` when an **AWS service is making the call on behalf of a principal** (e.g., CloudFormation calling IAM on your behalf). Setting this to `false` restricts actions to **direct human/application calls only**, preventing services from acting autonomously. Useful for detecting automation abuse.

Q74. A policy uses `"StringLike": {"s3:prefix": "home/${aws:username}/*"}`. What does this accomplish?

- A) Allows access to all prefixes starting with "home/"
- B) Creates dynamic, per-user home folder access in S3 using policy variables
- C) Restricts access to the IAM username's group prefix
- D) Allows access only when the username contains "home"

Answer: B Reasoning: `${aws:username}` is an **IAM policy variable** that dynamically resolves to the IAM username at evaluation time. This creates a **per-user S3 prefix** (like `home/alice/`) without needing separate policies per user. This is the recommended pattern for multi-user S3 home directories using a single scalable policy.

Q75. Can an IAM policy **Allow** access to a resource and also **Deny** access to the same resource within the same policy document?

- A) No — conflicting statements in one policy are invalid
- B) Yes — and the Deny always wins
- C) Yes — and the Allow always wins (most recent statement)
- D) No — IAM parses only the last statement per action

Answer: B Reasoning: A single policy can contain both Allow and Deny statements for the same action/resource. IAM evaluates **all statements** and the Deny wins regardless of ordering. This is used intentionally — e.g., Allow `s3:*` on `*` but Deny `s3:DeleteBucket` on `*` in the same policy to give broad access with a specific carve-out.

SECTION 7: IAM Roles for Services & Advanced Patterns (Q76–Q88)

Q76. An EC2 instance has an instance profile. The application on the instance makes API calls. Where do the credentials come from?

- A) Environment variables set at AMI creation
- B) Instance metadata service (IMDS) at `169.254.169.254`
- C) AWS Secrets Manager
- D) A hardcoded access key in the AMI

Answer: B Reasoning: EC2 instance profiles deliver **temporary credentials** via the **Instance Metadata Service (IMDS)** at the link-local address `169.254.169.254/latest/meta-data/iam/security-credentials/`. AWS SDKs automatically retrieve and rotate these credentials using the credential provider chain.

Q77. A security team wants to prevent EC2 instances from accessing IMDS to retrieve credentials. What should they configure?

- A) Disable the instance profile
- B) Set `HttpEndpoint: disabled` in the instance metadata options
- C) Block `169.254.169.254` in the security group
- D) Use IMDSv2 only

Answer: B Reasoning: Setting `HttpEndpoint: disabled` in the **instance metadata options** completely disables the IMDS, preventing any credential retrieval via the metadata service. Security groups cannot block the IMDS because it's a **link-local service** (not a real network hop). IMDSv2 adds session-oriented security but doesn't disable IMDS.

Q78. What is the key security improvement of **IMDSv2** over IMDSv1?

- A) IMDSv2 uses HTTPS; IMDSv1 uses HTTP
- B) IMDSv2 requires a session token obtained via a PUT request, preventing SSRF attacks from stealing credentials
- C) IMDSv2 encrypts credentials at rest
- D) IMDSv2 adds MFA for metadata access

Answer: B Reasoning: IMDSv2 requires a **session-oriented token** obtained via an HTTP **PUT request**. SSRF (Server-Side Request Forgery) attacks typically use GET requests, so they cannot initiate the PUT to get the token. IMDSv1 only requires a GET, making it vulnerable to SSRF-based credential theft. AWS now recommends IMDSv2 for all EC2 instances.

Q79. A Lambda function's execution role has `dynamodb:GetItem`. The Lambda is invoked by S3 (event notification). Can S3 invoke Lambda without additional permissions?

- A) Yes — S3 can invoke any Lambda automatically
- B) No — S3 needs an IAM role to invoke Lambda
- C) Yes — but only if a Lambda resource-based policy allows `s3.amazonaws.com` to invoke it
- D) No — Lambda invocations always require IAM roles

Answer: C Reasoning: For S3 to invoke Lambda, a **Lambda resource-based policy** must allow `s3.amazonaws.com` as the principal with `lambda:InvokeFunction`. The policy should also include a Condition with `aws:SourceArn` (the specific bucket ARN) to prevent confused deputy attacks. S3 does not assume a role; it uses the resource policy.

Q80. An ECS task needs to access DynamoDB. Where should the IAM permissions be configured?

- A) On the EC2 instance profile running the ECS agent
- B) On the ECS task definition's task role
- C) On the ECS service's IAM role
- D) On the ECS task definition's task execution role

Answer: B Reasoning: ECS has two distinct role types: (1) **Task Execution Role** — used by the ECS agent to pull container images and write logs (manages infrastructure). (2) **Task Role** — used by the **application code** running inside the container to access AWS services (DynamoDB, S3, etc.). DynamoDB access should be on the **task role**, not the execution role.

Q81. A company uses EKS (Kubernetes). Pods need AWS permissions. What is the recommended approach?

- A) Assign an EC2 instance profile to all worker nodes
- B) Use IAM Roles for Service Accounts (IRSA) — associate an IAM role with a Kubernetes service account via OIDC
- C) Embed IAM user credentials in Kubernetes secrets
- D) Use AWS Systems Manager Parameter Store for credentials

Answer: B Reasoning: IRSA (IAM Roles for Service Accounts) is the AWS-recommended approach. Each Kubernetes service account is associated with an IAM role via an **OIDC identity provider**. Only pods using that service account get the associated IAM permissions. This provides pod-level granularity, unlike node-level instance profiles which grant permissions to all pods on a node.

Q82. A service-linked role is created automatically by AWS when you enable certain services. What is unique about service-linked roles?

- A) They cannot have permissions boundaries applied
- B) They can only be deleted after the associated service no longer requires them
- C) They are shared across all AWS accounts in an organization
- D) They grant root-level access to the service

Answer: B Reasoning: Service-linked roles are predefined by AWS for specific services and cannot be deleted while the associated service is using them. For example, the `AWSServiceRoleForElasticLoadBalancing` role can only be deleted after removing all load balancers. They also cannot be freely edited — only the trust policy can sometimes be modified by AWS.

Q83. An IAM policy contains `"Resource": "arn:aws:s3::my-bucket"` for `s3:PutObject`. A user tries to upload an object. Does this work?

- A) Yes — the bucket ARN covers all objects inside it
- B) No — `s3:PutObject` is an object-level action and requires `arn:aws:s3::my-bucket/*`
- C) Yes — S3 automatically expands bucket ARNs to include objects
- D) No — you need both the bucket and object ARNs in a list

Answer: B Reasoning: S3 has a critical ARN distinction: `arn:aws:s3::my-bucket` is the bucket itself. `arn:aws:s3::my-bucket/*` covers objects in the bucket. Object-level actions (`PutObject`, `GetObject`, `DeleteObject`) must use the object ARN with `/*`. Bucket-level actions (`ListBucket`, `GetBucketAcl`) use the bucket ARN. Using the bucket ARN for `PutObject` will result in an implicit Deny.

Q84. AWS IAM Access Analyzer generates findings for which type of access?

- A) Overly permissive identity-based policies
- B) Resources accessible outside of your account or organization (external access)
- C) IAM users without MFA enabled
- D) Roles with unused permissions

Answer: B Reasoning: IAM Access Analyzer identifies resources in your account (S3 buckets, IAM roles, KMS keys, SQS queues, Lambda functions, Secrets Manager secrets) that can be accessed by **principals outside your account or organization**. For analyzing unused permissions, use **IAM Access Advisor**. For overly permissive policies, use **Access Analyzer policy validation**.

Q85. A company wants to ensure IAM users use only the permissions they actually need. Which tool provides a report on last-used services and actions?

- A) AWS Config
- B) AWS CloudTrail
- C) IAM Access Advisor (Last Accessed Information)
- D) AWS Security Hub

Answer: C Reasoning: IAM Last Accessed Information (Access Advisor) shows when IAM entities last accessed each AWS service. This enables **least-privilege enforcement** by identifying unused permissions. AWS also provides **CloudTrail-based access analysis** in Access Analyzer for action-level granularity. CloudTrail records all API calls but doesn't aggregate last-used data.

Q86. What is an **IAM condition operator** for checking if a value exists in a multi-value request parameter?

- A) `StringEquals`
- B) `ForAllValues:StringEquals`
- C) `StringLike`
- D) `ArnEquals`

Answer: B Reasoning: `ForAllValues:StringEquals` is used when a condition key can have **multiple values** (e.g., `aws:TagKeys`). It evaluates that ALL provided values match one of the expected values. The counterpart

`ForAnyValue:StringEquals` checks if ANY of the provided values match. These are **set operators** essential for tag-based policies.

Q87. What is the maximum number of managed policies that can be attached to a single IAM role?

- A) 5
- B) 10
- C) 20
- D) 50

Answer: B Reasoning: AWS allows a maximum of **10 managed policies** attached to a single IAM user, group, or role. You can also add inline policies, but those have character limits. If you need more than 10 policies, you should consolidate them into fewer customer-managed policies.

Q88. A company creates an IAM role for a third-party vendor. The vendor is in a different AWS account. The vendor's account ID changes. What is the minimum change required to maintain access?

- A) Delete and recreate the role
- B) Update the role's trust policy with the new account ID
- C) Update the role's permissions policy
- D) Create a new IAM user for the vendor

Answer: B Reasoning: The **trust policy** (`AssumeRolePolicyDocument`) specifies who can assume the role. Changing the trusted account only requires updating the trust policy — not the permissions policy, and certainly not recreating the role. This is a maintenance operation that preserves all existing permissions.

SECTION 8: Security, Compliance & Best Practices (Q89–100)

Q89. An organization wants to enforce MFA for all IAM users accessing the AWS console. The most scalable approach is:

- A) Manually enable MFA for each user
- B) Attach an IAM policy that denies all actions except `iam:CreateVirtualMFADevice` and `iam:EnableMFADevice` unless `aws:MultiFactorAuthPresent` is true
- C) Use AWS Config to detect non-MFA access and auto-remediate
- D) Enable MFA in the Account Settings console

Answer: B Reasoning: The scalable approach is to attach a **policy to all users/groups** that Denies all actions (except MFA setup actions) when `aws:MultiFactorAuthPresent` is **false**. This forces users to set up and use MFA before they can do anything. AWS provides the `AWSVirtualMFADevice` managed policy for this pattern. AWS Config detects but doesn't enforce in real-time.

Q90. What is the AWS recommended way to handle access keys for applications running on EC2?

- A) Use long-term IAM user access keys stored in environment variables
- B) Store keys in AWS Secrets Manager and retrieve at runtime
- C) Use IAM roles with instance profiles — no long-term credentials needed
- D) Encrypt access keys using KMS and store them in S3

Answer: C Reasoning: The AWS best practice is to **never use long-term access keys on EC2**. Instead, use **IAM roles with instance profiles**, which automatically provide short-lived, auto-rotating temporary credentials via IMDS. This eliminates key management, rotation challenges, and the risk of credential exposure.

Q91. An IAM user's access key is compromised. What are the correct immediate steps?

- A) Delete the IAM user immediately
- B) Rotate the access key, identify unauthorized activity via CloudTrail, revoke active sessions, and review/tighten permissions
- C) Change the IAM user's password
- D) Enable MFA on the account

Answer: B Reasoning: The response to a compromised access key involves: (1) **Deactivate/delete** the compromised key, (2) Create a new key if needed, (3) Review **CloudTrail** logs for unauthorized activity, (4) Revoke any active sessions (via **sts:RevokeSession** or attaching a Deny-all policy with a time condition), (5) Check if any resources were created/modified. Simply changing the password does nothing for access keys.

Q92. How does **IAM Identity Center (SSO)** differ from traditional IAM federation?

- A) Identity Center only works with AWS-managed identity providers
- B) Identity Center provides centralized access management across multiple AWS accounts and applications with a single sign-on experience, using short-lived credentials
- C) Identity Center replaces IAM roles with IAM groups for multi-account access
- D) Identity Center requires SAML; IAM federation supports OIDC

Answer: B Reasoning: IAM Identity Center (formerly AWS SSO) provides centralized access across all accounts in an AWS Organization with a single portal. It uses **permission sets** (templates that create IAM roles) and provides temporary credentials. Traditional IAM federation requires per-account configuration. Identity Center is the AWS-recommended approach for multi-account access management.

Q93. A permission set in IAM Identity Center is updated. What happens to active sessions of users who are already logged in?

- A) Active sessions immediately reflect the new permissions
- B) Active sessions maintain old permissions until they expire; new sessions get updated permissions
- C) All active sessions are immediately terminated
- D) Users must re-authenticate within 5 minutes

Answer: B Reasoning: IAM Identity Center uses **temporary credentials with a session duration**. When a permission set is updated, the **underlying IAM roles are updated**, but **existing STS sessions are not revoked**. Users with active credentials continue using old permissions until their session expires. To immediately enforce changes, you must revoke active sessions.

Q94. An IAM role is created for AWS Lambda with `AmazonDynamoDBFullAccess`. A security review flags this. What is the recommended remediation?

- A) Replace with a custom policy granting only the specific DynamoDB actions and table ARNs the function needs
- B) Use `AmazonDynamoDBReadOnlyAccess` instead
- C) Attach a permissions boundary limiting to DynamoDB only
- D) Remove the policy and use resource-based policies on DynamoDB

Answer: A Reasoning: The principle of **least privilege** requires granting only the specific actions on specific resources. `AmazonDynamoDBFullAccess` is overly broad (all DynamoDB actions on all tables). A custom policy should specify only required actions (e.g., `dynamodb:GetItem`, `dynamodb:PutItem`) on the specific table ARN. Option B is still too broad; Option C doesn't add granularity; Option D is impossible (DynamoDB doesn't support resource-based policies).

Q95. What does the **AWS Root Account** have that IAM users and roles do not?

- A) Ability to create other IAM users
- B) Access to certain account-level tasks that cannot be delegated: closing the account, changing the support plan, enabling MFA delete on S3, subscribing/unsubscribing from some services
- C) Unlimited policy size
- D) Automatic administrator access in all member accounts of an organization

Answer: B Reasoning: The root account has exclusive access to certain **account-level tasks**: closing the account, changing payment methods, altering the root user's email/password, viewing certain billing information, enabling IAM access to billing, enabling S3 MFA Delete, subscribing to GovCloud. These cannot be granted to IAM users or roles.

Q96. An IAM policy uses `"Resource": "arn:aws:iam::*:role/MyRole"`. What does the `*` in the account position mean?

- A) The role in any AWS account
- B) A syntax error — account IDs cannot be wildcards in IAM policies
- C) All roles named `MyRole` in the current account
- D) All roles across the organization

Answer: A Reasoning: In resource ARNs within IAM policies, you can use `*` in the account ID field to mean **any account**. `arn:aws:iam::*:role/MyRole` refers to a role named `MyRole` in **any AWS account**. This is used in trust policies or `sts:AssumeRole` policies to allow assuming a same-named role across multiple accounts — a common multi-account pattern.

Q97. What is the purpose of the `aws:SourceAccount` condition key when used alongside `aws:SourceArn`?

- A) `aws:SourceAccount` is redundant if `aws:SourceArn` is set
- B) Some services (like S3 notifications) don't always populate `aws:SourceArn`; `aws:SourceAccount` provides an additional account-level check to prevent confused deputy attacks
- C) `aws:SourceAccount` restricts which region the source is in
- D) They serve the same purpose and only one should be used

Answer: B Reasoning: For services like S3 that may not always populate `aws:SourceArn` fully, using both `aws:SourceArn` (when available) and `aws:SourceAccount` provides defense in depth against the **confused deputy problem**. AWS documentation recommends using both conditions together in resource-based policies for services like SNS, S3, and CloudWatch.

Q98. A company wants to implement break-glass access for emergencies. An IAM role named `BreakGlassAdmin` exists. How should access be logged and alerted?

- A) Enable CloudTrail and create a CloudWatch alarm on `sts:AssumeRole` for the `BreakGlassAdmin` role ARN
- B) Use AWS Config to detect role usage
- C) Enable S3 server access logs
- D) Use VPC Flow Logs to track role usage

Answer: A Reasoning: Break-glass access should be monitored via **CloudTrail** (which logs all `sts:AssumeRole` calls with the role ARN) and **CloudWatch Logs/Alarms** to trigger immediate alerts when the break-glass role is assumed. You can create a metric filter on the CloudTrail log group matching the specific role ARN and send an SNS alert. AWS Config detects configuration changes, not real-time API calls.

Q99. What is the correct way to allow an IAM user to view their own IAM user details (access keys, MFA devices) but NOT view other users?

- A) Grant `iam:GetUser` with `Resource: *`
- B) Grant `iam:GetUser`, `iam:ListAccessKeys`, and `iam:ListMFADevices` with Resource scoped to `arn:aws:iam::ACCOUNT:user/${aws:username}`
- C) Use IAM Access Advisor
- D) Grant read-only IAM access via `IAMReadOnlyAccess` managed policy

Answer: B Reasoning: Using the **IAM policy variable** `${aws:username}` in the resource ARN scopes IAM actions to only the **current user's own user object**. This is the self-service IAM pattern. `Resource: *` would expose all users' information. `IAMReadOnlyAccess` grants read access to all IAM resources — much too broad.

Q100. A new IAM user is created via CloudFormation. The CloudFormation service role has `iam:CreateUser`. The company wants to audit all IAM user creations. Which service provides the most real-time evidence of who created the user?

- A) AWS Config — detects IAM user resource creation
- B) IAM Credential Report
- C) AWS CloudTrail — logs the `CreateUser` API call with the caller's identity
- D) Amazon GuardDuty

Answer: C Reasoning: **CloudTrail** records every API call including `iam:CreateUser` with the **caller identity** (who made the call), timestamp, source IP, and parameters. This is the authoritative audit trail for IAM actions. AWS Config detects configuration changes (that a user now exists) but doesn't provide the API-level who/when/how context. CloudTrail is the correct answer for **auditing and accountability**.

Quick Reference Cheat Sheet

Topic	Key Rule
Explicit Deny	Always wins over any Allow

Implicit Deny	Default for all actions with no policy
Cross-Account S3	Both IAM policy + bucket policy required
KMS Key Policy	Must explicitly allow; IAM alone is insufficient
SCP	Guardrail only — not a grant; doesn't apply to management account
Permissions Boundary	Ceiling only — not a grant; intersects with identity policy
Session Policy	Can only restrict, never expand role permissions
IMDS Credentials	Retrieved from 169.254.169.254; IMDSv2 requires PUT first
Role Chaining	Max 1 hour session regardless of role maximum
<code>iam:PassRole</code>	Privilege escalation vector — treat as sensitive
S3 ARN	Bucket: <code>:::bucket</code> ; Objects: <code>:::bucket/*</code> — don't mix
ABAC	Uses <code>\${aws:PrincipalTag/key}</code> + <code>aws:ResourceTag/key</code>
STS Key Prefix	<code>ASIA</code> = temporary; <code>AKIA</code> = long-term
Policy Variables	<code>\${aws:username}</code> , <code>\${aws:user-id}</code> for dynamic scoping
Service-Linked Roles	Cannot delete while service is using them